

OncoImplexus User Guide: From VCF to Interactive Reports

Contents

1. Quick Start with Bundled Example (DO17373)	1
Complete Example with DO17373	1
2. Single Sample Workflow (VCF Input)	2
Step 1: Loading Data from VCF	3
Step 2: Preparing Gene Annotations	3
Option A: Using Bundled Annotations (Recommended)	3
Option B: Using Bioconductor TxDb (Advanced)	3
Step 3: Integrated Detection	3
Step 4: Generating the Interactive HTML Report	4
Complete VCF Example Script	4
3. Using Data Frames (Advanced)	5
Creating SVs Object from Data Frame	5
Creating CNVsegs Object from Data Frame	6
4. Cohort Analysis Workflow	6
Step 1: Batch Processing	6
Step 2: Aggregate & Summarize	7
5. Interpreting the Results	7
High vs. Low Confidence (Chromothripsis)	7
Likely vs. Possible (Chromoplexy)	8
Likely vs. Possible (Chromoanasythesis)	8
6. Troubleshooting FAQ	8

This guide provides a comprehensive walkthrough for analyzing cancer genomes using **OncoImplexus**, covering both single-sample clinical analysis and large-scale cohort studies.

1. Quick Start with Bundled Example (DO17373)

OncoImplexus includes a bundled example dataset (DO17373) from the original ShatterSeek publication. This sample exhibits chromothripsis on chromosomes 2, 21, and X, making it ideal for testing your installation.

Complete Example with DO17373

```
library(OncoImplexus)

# === Load Bundled Example Data ===
load(system.file("extdata", "DO17373.RData", package = "OncoImplexus"))

# === Create SVs Object ===
```

```

sv_obj <- SVs(
  chrom1 = SV_D017373$chrom1,
  pos1 = SV_D017373$start1,
  chrom2 = SV_D017373$chrom2,
  pos2 = SV_D017373$start2,
  strand1 = SV_D017373$strand1,
  strand2 = SV_D017373$strand2,
  SVtype = SV_D017373$svclass,
  sv_id = SV_D017373$sv_id
)

# === Create CNVsegs Object ===
cnv_obj <- CNVsegs(
  chrom = as.character(SCNA_D017373$chromosome),
  start = SCNA_D017373$start,
  end = SCNA_D017373$end,
  total_cn = SCNA_D017373$total_cn
)

# === Run Integrated Detection ===
results <- detect_chromoanagenesis(sv_obj, cnv_obj, genome = "hg19")

# === Load Gene Annotations ===
gene_granges <- readRDS(
  system.file("extdata", "hg19_genes.rds", package = "OncoImplexus")
)

# === Generate HTML Report ===
generate_interactive_report(
  result = results,
  SV.sample = sv_obj,
  CNV.sample = cnv_obj,
  output_file = "D017373_Report.html",
  output_dir = "reports",
  sample_name = "D017373",
  genome = "hg19",
  gene_granges = gene_granges
)

```

Expected Output:

- Overall classification: **Chromothripsis only**
- High confidence chromothripsis: **3 events** (chromosomes 2, 21, X)
- Possible chromoplexy: 277 chains
- Possible chromoanasythesis: 6 regions

2. Single Sample Workflow (VCF Input)

The most common use case is analyzing a single tumor sample from VCF files produced by SV/CNV callers.

Step 1: Loading Data from VCF

OncoImplexus supports direct VCF input. It automatically parses BND (breakend) and standard SV types.

```
library(OncoImplexus)

# Load SVs (supports Manta, Delly, GRIDSS, DRAGEN, Lumpy, Sniffles)
sv_data <- read_sv_vcf("tumor.sv.vcf.gz")

# Load CNVs (supports CNVkit, GATK, Control-FREEC, DRAGEN)
cnv_data <- read_cnv_vcf("tumor.cnv.vcf.gz")
```

Note: The VCF reader automatically detects the caller type and parses accordingly. The returned objects are already in the correct format (SVs and CNVsegs S4 classes).

Step 2: Preparing Gene Annotations

For gene fusion prediction and driver impact analysis, you can either use the **bundled gene annotations** or load from Bioconductor databases.

Option A: Using Bundled Annotations (Recommended)

OncoImplexus includes pre-built gene annotation files for hg19 and hg38:

```
# For hg38 genome
gene_granges <- readRDS(
  system.file("extdata", "hg38_genes.rds", package = "OncoImplexus")
)

# For hg19 genome
gene_granges <- readRDS(
  system.file("extdata", "hg19_genes.rds", package = "OncoImplexus")
)
```

Option B: Using Bioconductor TxDb (Advanced)

```
library(TxDb.Hsapiens.UCSC.hg38.knownGene)
library(org.Hs.eg.db)

txdb <- TxDb.Hsapiens.UCSC.hg38.knownGene
genes_gr <- genes(txdb)
symbols <- mapIds(org.Hs.eg.db, keys = names(genes_gr),
  column = "SYMBOL", keytype = "ENTREZID")
mcols(genes_gr)$symbol <- symbols
gene_granges <- genes_gr[!is.na(mcols(genes_gr)$symbol)]
```

Step 3: Integrated Detection

Run all three mechanisms (Chromothripsis, Chromoplexy, Chromoanagenesis) in one call:

```
results <- detect_chromoanagenesis(
  sv_data,
  cnv_data,
  genome = "hg38" # or "hg19"
)
```

Step 4: Generating the Interactive HTML Report

This is the final deliverable, producing a clinical-grade dashboard with visualizations.

```
generate_interactive_report(  
  result = results,  
  SV.sample = sv_data,  
  CNV.sample = cnv_data,  
  output_file = "Patient_X_Report.html",  
  output_dir = "reports",  
  sample_name = "Patient X (Tumor)",  
  genome = "hg38",  
  gene_granges = gene_granges  
)
```

Important Parameters:

Parameter	Description
result	The output from <code>detect_chromoanagenesis()</code>
SV.sample	The SVs object from <code>read_sv_vcf()</code>
CNV.sample	The CNVsegs object from <code>read_cnv_vcf()</code>
output_file	Output HTML filename (just the filename, not path)
output_dir	Directory to save the report
sample_name	Display name in the report
genome	Reference genome ("hg19" or "hg38")
gene_granges	Gene annotation GRanges object

Complete VCF Example Script

Here's a complete working example using VCF files:

```
library(OncoImplexus)  
  
# === Configuration ===  
sample_id <- "H435"  
sv_vcf <- "path/to/H435.sv.vcf.gz"  
cnv_vcf <- "path/to/H435.cnv.vcf.gz"  
output_dir <- "reports"  
genome <- "hg38"  
  
# === Step 1: Load Data ===  
sv_data <- read_sv_vcf(sv_vcf)  
cnv_data <- read_cnv_vcf(cnv_vcf)  
  
# === Step 2: Load Gene Annotations ===  
gene_granges <- readRDS(  
  system.file("extdata", "hg38_genes.rds", package = "OncoImplexus")  
)  
  
# === Step 3: Run Analysis ===  
results <- detect_chromoanagenesis(sv_data, cnv_data, genome = genome)  
  
# === Step 4: Generate Report ===  
generate_interactive_report(  
  result = results,
```

```
SV.sample = sv_data,
CNV.sample = cnv_data,
output_file = paste0(sample_id, "_Report.html"),
output_dir = output_dir,
sample_name = sample_id,
genome = genome,
gene_granges = gene_granges
)
```

3. Using Data Frames (Advanced)

If you have SV/CNV data in custom formats (e.g., from non-standard callers or downstream processing), you can create the S4 objects manually.

Creating SVs Object from Data Frame

```
# Your SV data frame must have these columns
sv_df <- data.frame(
  chrom1 = c("1", "2", "1"),
  pos1 = c(1000000, 5000000, 8000000),
  chrom2 = c("1", "3", "1"),
  pos2 = c(2000000, 8000000, 9000000),
  strand1 = c("+", "-", "+"),
  strand2 = c("-", "+", "+"),
  SVtype = c("DEL", "TRA", "h2hINV"),
  sv_id = c("SV1", "SV2", "SV3")
)

# Create SVs object
sv_obj <- SVs(
  chrom1 = sv_df$chrom1,
  pos1 = sv_df$pos1,
  chrom2 = sv_df$chrom2,
  pos2 = sv_df$pos2,
  strand1 = sv_df$strand1,
  strand2 = sv_df$strand2,
  SVtype = sv_df$SVtype,
  sv_id = sv_df$sv_id
)
```

SV Types:

SVtype	Description	Strand Pattern
DEL	Deletion-like	+/-
DUP	Duplication-like	-/+
h2hINV	Head-to-head inversion	+/+
t2tINV	Tail-to-tail inversion	-/-
TRA	Translocation	varies

Creating CNVsegs Object from Data Frame

```
# Your CNV data frame must have these columns
cnv_df <- data.frame(
  chr = c("1", "1", "2"),
  start = c(1, 5000000, 1),
  end = c(4999999, 10000000, 50000000),
  cn = c(2, 1, 3)
)

# Create CNVsegs object
cnv_obj <- CNVsegs(
  chrom = cnv_df$chr,
  start = cnv_df$start,
  end = cnv_df$end,
  total_cn = cnv_df$cn
)
```

Note: Chromosome names must be in Ensembl notation (1-22, X, Y) without “chr” prefix.

4. Cohort Analysis Workflow

For research studies involving multiple patients, follow the batch processing pipeline.

Step 1: Batch Processing

Process a list of samples using a loop:

```
library(OncoImplexus)

sample_list <- c("SP1001", "SP1002", "SP1003")
data_dir <- "vcf_files"
output_dir <- "cohort_results"

# Create output directory
dir.create(output_dir, showWarnings = FALSE, recursive = TRUE)

# Load gene annotations once
gene_granges <- readRDS(
  system.file("extdata", "hg19_genes.rds", package = "OncoImplexus")
)

for (sid in sample_list) {
  message("Processing: ", sid)

  # Load data
  sv_data <- read_sv_vcf(file.path(data_dir, paste0(sid, ".sv.vcf.gz")))
  cnv_data <- read_cnv_vcf(file.path(data_dir, paste0(sid, ".cnv.vcf.gz")))

  # Run detection
  results <- detect_chromoanagenesis(sv_data, cnv_data, genome = "hg19")

  # Save results
```

```

saveRDS(results, file = file.path(output_dir, paste0(sid, ".rds")))

# Generate individual reports
generate_interactive_report(
  result = results,
  SV.sample = sv_data,
  CNV.sample = cnv_data,
  output_file = paste0(sid, "_Report.html"),
  output_dir = output_dir,
  sample_name = sid,
  genome = "hg19",
  gene_granges = gene_granges
)
}

```

Step 2: Aggregate & Summarize

Use the built-in summary tool to create a master CSV table:

```

summary_table <- summarize_cohort_results(
  results_dir = "cohort_results",
  output_csv = "Study_Summary.csv"
)

# View summary metrics
head(summary_table)

```

Master Summary Table Columns:

Column	Description
sample_id	Sample name (from .rds filename)
classification	Integrated diagnosis (e.g., “Multiple mechanisms”)
ct_hc	High Confidence chromothripsis events
cp_likely	Likely chromoplexy chains
cs_likely	Likely chromoanagenesis regions
n_fusions	Predicted gene fusions

Example Output:

sample_id	classification	ct_hc	cp_likely	cs_likely	n_fusions
PATIENT_01	Chromothripsis only	2	0	0	8
PATIENT_02	Multiple mechanisms	1	42	0	15
PATIENT_03	No chromoanagenesis	0	0	0	0

5. Interpreting the Results

High vs. Low Confidence (Chromothripsis)

- **High Confidence:** Passes the “Fragment Joins Test” ($p > 0.05$) and has strong CN oscillations (≥ 7 segments). This is the “gold standard” for chromothripsis.

- **Low Confidence:** Clustered SVs with fewer CN oscillations (4-6). May represent early or degraded events.

Likely vs. Possible (Chromoplexy)

- **Likely:** Chains with 3+ chromosomes and high CN stability.
- **Possible:** Chains that are borderline or involve only 2 chromosomes.

Likely vs. Possible (Chromoanasyntesis)

- **Likely:** Regions with clear copy number gradients AND supporting SV topology evidence.
- **Possible:** Regions with partial evidence (gradient only or topology only).

6. Troubleshooting FAQ

Q: Why does my report say ‘N/A’ for DNA Repair?

A: Ensure the corresponding BSgenome package is installed:

```
BiocManager::install("BSgenome.Hsapiens.UCSC.hg38") # for hg38
BiocManager::install("BSgenome.Hsapiens.UCSC.hg19") # for hg19
```

Q: I get ‘SV.sample must be a SVs object’ error.

A: Make sure you’re using an SVs object, not a data frame. If reading from VCF, `read_sv_vcf()` returns the correct object. If using custom data, create the object manually (see Section 3).

Q: I get ‘cannot open connection’ errors during report rendering.

A: This usually happens if the data is too large. OncoImplexus v1.0.0+ uses direct environment injection to solve this. Ensure you are using the latest version.

Q: My SV caller is not recognized.

A: The VCF reader auto-detects most callers (Manta, Delly, GRIDSS, DRAGEN, Lumpy, Sniffles). If your caller is not recognized, create the SVs object manually from a data frame (see Section 3).

Q: How do I suppress browser tabs opening during report generation?

A: This is fixed in v1.0.0+. The report template now includes options to prevent automatic browser opening.

Q: Chromosome name errors?

A: OncoImplexus uses Ensembl chromosome notation (1-22, X, Y) without “chr” prefix. The VCF readers handle this automatically, but if using custom data, ensure chromosomes are named correctly.

Q: How do I analyze hg19 vs hg38 samples?

A: Simply specify the correct genome parameter and use the matching gene annotations:

```
# For hg38
gene_gr_38 <- readRDS(system.file("extdata", "hg38_genes.rds",
                                package = "OncoImplexus"))
results <- detect_chromoanagenesis(sv_data, cnv_data, genome = "hg38")

# For hg19
gene_gr_19 <- readRDS(system.file("extdata", "hg19_genes.rds",
                                package = "OncoImplexus"))
results <- detect_chromoanagenesis(sv_data, cnv_data, genome = "hg19")
```